

SESIÓN 2 : MySQL + JPA básico (1 tabla)

"Ahora los datos tienen que sobrevivir"

OBJETIVO DE LA SESIÓN

Hasta ahora:

- La API funciona
- Pero los datos están en memoria
- Al reiniciar → se pierden

 **Problema real:**

"Una API sin persistencia no sirve para nada real."

Objetivo:

 Guardar y leer datos desde MySQL usando JPA, sin complicar nada.

TEORÍA CLAVE

1 ¿Por qué JPA?

- No queremos escribir SQL todo el día
- Queremos trabajar con objetos Java
- JPA traduce objetos ↔ tablas

 **Mensaje:**

"Vosotros pensáis en objetos, JPA piensa en SQL."

2 ¿Qué es una Entity?

- Representa una fila de una tabla
- Vive en el paquete entity
- NO es la API, es el modelo de datos

 **Importante:**

Entity ≠ lo que devuelvo al cliente (eso vendrá con DTOs más adelante)

3 Repository

- Capa que habla con la BD
- CRUD sin escribir SQL
- Spring genera la implementación



ESTRUCTURA DE PROYECTO (desde ahora)

```
com.example.books
├── controller
├── entity    ← NUEVO (antes model)
├── repository
└── BooksApplication
```

📌 **Cambio importante:** El paquete `model` ahora se llama `entity` porque representa entidades de base de datos.



BLOQUE 1 — Configuración MySQL

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/books_db
spring.datasource.username=root
spring.datasource.password=secret

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

springdoc.swagger-ui.path=/docs
```

Explicación

ddl-auto=update

- crea/actualiza tablas automáticamente
- solo para desarrollo

show-sql=true

- ver SQL real → entender qué hace JPA

💡 Mensaje clave:

"JPA no es magia: genera SQL."



BLOQUE 2 — Entity Book

📁 entity/Book.java

```
package com.example.books.entity;

import jakarta.persistence.*;

/**
 * Entity = representación de una tabla en BD.
 * Cada instancia = una fila.
 */
@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // clave primaria

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false)
    private String category;

    // JPA necesita constructor vacío
    protected Book() {
    }

    public Book(String title, String author, String category) {
        this.title = title;
        this.author = author;
        this.category = category;
    }

    // getters y setters (JPA los necesita)
    public Long getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    public String getCategory() {
        return category;
    }

    public void setCategory(String category) {
        this.category = category;
    }
}
```



@Entity → JPA la gestiona



@Id → clave primaria



GenerationType.IDENTITY → la BD genera el id



Constructor vacío → obligatorio



No hay lógica aquí → solo datos



Mensaje clave:

"Las entities representan datos, no reglas."

BLOQUE 3 — Repository

 repository/BookRepository.java

```
package com.example.books.repository;

import com.example.books.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;

/**
 * JpaRepository nos da:
 * - save
 * - findAll
 * - findById
 * - deleteById
 * sin escribir SQL
 */
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

Teoría importante

Spring crea la implementación en runtime

No se programa "cómo", solo "qué"

Patrón Repository

 **Mensaje clave:**

"Nosotros declaramos intenciones, Spring implementa."



BLOQUE 4 — Refactor del Controller

👉 Quitamos la lista en memoria

👉 Usamos MySQL

📁 controller/BookController.java

```
package com.example.books.controller;

import com.example.books.entity.Book;
import com.example.books.repository.BookRepository;
import org.springframework.web.bind.annotation.*;

import java.util.List;

/**
 * Controller REST básico con persistencia.
 * Todavía NO usamos ResponseEntity (eso es la sesión 3).
 */
@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;

    // Inyección por constructor (buena práctica)
    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    // READ all
    @GetMapping
    public List<Book> getAll() {
        return bookRepository.findAll();
    }

    // READ by id
    @GetMapping("/{id}")
    public Book getById(@PathVariable Long id) {
        // AÚN frágil: lo mejoraremos en la sesión 3
        return bookRepository.findById(id).orElse(null);
    }

    // CREATE
    @PostMapping
    public Book create(@RequestBody Book book) {
        return bookRepository.save(book);
    }

    // UPDATE
    @PutMapping("/{id}")
    public Book update(@PathVariable Long id, @RequestBody Book updated) {
        Book existing = bookRepository.findById(id).orElse(null);
        if (existing == null) {
            return null; // intencionalmente mal (para la siguiente sesión)
        }
        existing.setTitle(updated.getTitle());
        existing.setAuthor(updated.getAuthor());
        existing.setCategory(updated.getCategory());
        return bookRepository.save(existing);
    }

    // DELETE
    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) {
        bookRepository.deleteById(id);
    }
}
```

BLOQUE 5 — Probar en Swagger

Pruebas guiadas:

01	02	03
POST /api/books	Reiniciar app	GET /api/books 👉 los datos siguen ahí

💡 Mensaje clave:

"Ahora la API ya es real."

CIERRE DE LA SESIÓN

Qué sabemos hasta ahora

-  Spring Boot REST
-  MySQL
-  JPA
-  Entity / Repository
-  CRUD persistente

Qué NO sabemos (y está bien)

-  status codes correctos
-  errores bien gestionados
-  validación
-  DTOs

👉 Eso es la SESIÓN 3